

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 2.1

Stream a Response from a Node.js Script

Module	2 — Building Agents with the Claude API & Agent SDK
Topic	2.1 — Anthropic Messages API Deep Dive
Estimated time	20–30 minutes
Tools needed	Terminal, Node.js 20+, a code editor, an active Claude API key
Prerequisite	Module 1 complete; <code>ANTHROPIC_API_KEY</code> available; Topic 2.1 briefing on the program website

What You're Building

GOAL

A Node.js script that opens a **streaming** request to the Claude Messages API, prints each chunk of text to the terminal as it arrives (not all at once), and prints the final input/output token usage once the stream completes.

This is the streaming variant of the request shape you first used in Module 1 — same `messages` array, same system prompt, but read incrementally instead of waiting for the full reply.

Steps 1–3: Set Up the Project

1 Create a project folder

In a terminal, outside of any other project:

TERMINAL

```
mkdir stream-lab
cd stream-lab
npm init -y
```

What you'll see: a new folder containing a `package.json` file.

2 Store your API key safely

Create a file named `.env` in the project folder with your Anthropic API key:

.ENV

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Before you go further: create a `.gitignore` file containing just the line `.env` — this key should never end up committed to git, even in a lab exercise.

3 Install the SDK and enable ESM

TERMINAL

```
npm install @anthropic-ai/sdk dotenv
```

Open `package.json` and add `"type": "module"` at the top level so you can use modern `import` syntax.

What you'll see: `@anthropic-ai/sdk` and `dotenv` listed under `dependencies` in `package.json`.

Steps 4–5: Write the Streaming Script

4 Open Claude Code and describe the task

In your project folder, run `claude` to start Claude Code, then give it this spec:

PROMPT TO CLAUDE CODE

```
Build a Node.js script (ESM, call it stream-response.js) that:
1. Loads ANTHROPIC_API_KEY from a .env file via dotenv, and exits
   with a clear error message (not a raw stack trace) if it's missing
2. Opens a STREAMING request to the Claude Messages API
   (model: "claude-sonnet-5", max_tokens: 1024) with the user message
   "Write a short, upbeat product description for an insulated
   water bottle."
3. Writes each text chunk to stdout as it arrives, with no buffering
   delay – the reader should see text appear incrementally, not all
   at once
4. After the stream finishes, prints the final input_tokens and
   output_tokens usage from the completed message on their own line

Use the @anthropic-ai/sdk package's streaming helper
(client.messages.stream), not a plain messages.create() call.
```

5 Review the diff before accepting

Claude Code will propose creating `stream-response.js` and updating `package.json`. Before accepting, check:

Check	Why it matters
Does it load the key via <code>dotenv</code> , not a hardcoded string?	Keeps the real key out of your source code
Does it use <code>client.messages.stream()</code> , not <code>messages.create()</code> ?	A non-streaming call defeats the point of this lab
Does it write chunks directly as they arrive (e.g. a "text" event handler), rather than collecting the full reply first?	That's the difference between "streaming" and just buffering then printing
Does it read usage from the <i>final</i> resolved message, not from an individual chunk?	Token counts are only accurate once the stream is complete
Does a missing API key print a clear message instead of an unhandled exception?	Matches the spec, and mirrors the reliability habits from Topic 2.4

If anything's missing, don't fix it yourself — ask Claude Code directly: "switch this to the streaming helper and write each chunk to stdout as it arrives." That request-and-iterate step is the point of the exercise.

Steps 6–7: Run and Verify

6 Install dependencies and run it

TERMINAL

```
npm install
node stream-response.js
```

What you'll see: the product description printing to the terminal in visible chunks — a few words at a time rather than the whole paragraph appearing at once — followed by a final line reporting input/output token counts.

7 Confirm the acceptance checklist

- ☐ Text appears incrementally in the terminal, not all at once
- ☐ The script reads `ANTHROPIC_API_KEY` from the environment, never hardcoded
- ☐ Final usage (`input_tokens` / `output_tokens`) is printed after the stream ends

Troubleshooting

Symptom	Likely cause	Try this
<code>ANTHROPIC_API_KEY</code> is undefined	<code>.env</code> not loaded, or in the wrong folder	Confirm <code>.env</code> sits next to <code>stream-response.js</code> ; check <code>dotenv.config()</code> runs before the client is created
Whole response prints at once, not streaming	Using <code>messages.create()</code> instead of <code>messages.stream()</code>	Ask Claude Code to switch to the streaming helper and write chunks via a <code>"text"</code> event handler
No usage numbers printed, or they look wrong	Reading usage from a partial chunk instead of the final message	Usage is only complete on the resolved value of <code>stream.finalMessage()</code>
<code>node: command not found</code>	Node not installed, or an old cached version on PATH	Run <code>node --version</code> ; should be 20+
API call fails with a 401 error	Key not valid or not loaded correctly	Re-check the value in <code>.env</code> ; regenerate it in the Anthropic Console if unsure

Where This Goes Next

Streaming is what makes the React chat widget from Module 1 (Lab 1.6) feel responsive once it's wired to a real backend later in the program — the reader sees text as it's generated instead of waiting on a spinner. It's also the same request shape every tool-use call in the rest of Module 2 builds on, just without a `tools` array attached yet.

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc. · Module 2: Building Agents with the Claude API & Agent SDK · Full module content: see the program website, Module 2 page.